One proof per mechanism

Shared implication proofs and a checked candidate scheduler
for the 215-state ZFC-consistency machine

A source-pinned proposal and implementation packet for ProveIt

22 October 2026

Abstract

The immediate recommendation is to share the implication derivation across its numeric and unrestricted interpretations, then share the candidate-search argument across enumerators. These are mathematically independent changes: the first provides a substantial source reduction; the second reduces duplicated reasoning and weakens the scheduler's assumptions, with little immediate aggregate size gain. The supplied Lean patch reduces the two implication proof files from 811 to 401 lines. The scheduler patch replaces sixteen separately maintained inductive proofs in four Lean enumerators by three shared inductive proofs. A constructive Rocq scheduler and current-source adapters are also supplied.

The packet contains complete source overlays, reviewable patches, reproduction scripts, source hashes, compiler logs, and a checked counterexample to an unsafe quantifier weakening. It builds on ideas and derivations already present in the repository, explicitly distinguishing their reuse from the new integration and verification work. Component checks are fresh; a new complete physical-machine endpoint build is not claimed. Machine tables and the existing consistency statement are outside the proposed changes.

1 Recommendation and exact scope

Adopt the shared implication basis first. It is the clearest immediate win: one derivation serves both the dominated numeric calculus and the unrestricted implicational calculus. The supplied patch preserves the public interfaces, includes the maintained producer, and has been checked against the actual retained sources.

Adopt the generic scheduler for reuse across candidates. Its contract mentions only a candidate index, a decreasing pending-work rank, and two successful-step laws. This isolates the mathematical search argument from the encoding and the prover state. For a standalone K215-only source-size target, retaining the smaller concrete argument is a defensible choice until more clients share the core or obsolete compatibility wrappers can be retired.

Then address decoder certificates. A route and its internal decoder clock should be selected before the tape state. This is a promising second stage, already identified by earlier repository work. It needs its own implementation and full coverage checks; it is not part of the checked patches in this packet.

1.1 Pinned baseline, existing work, and measurement boundary

All comparisons use repository commit

`57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`.

The actual phase-twelve endpoint imports 179 project modules, comprising 2,129,439 source bytes; the packet independently inventories that closure. Those are baseline measurements, not the results of the component refactor [2]. In particular, a recent commit message is not a reliable description of the files retained at this revision.

The six-goal shared implication basis already appears in commits `3a7a61ee` and `49e99c5b`. The retained phase-twelve proof files still use the older three-goal generic core and the separate unrestricted derivation. This packet reconciles the existing shared basis with that actual baseline, synchronizes its producer, and freshly checks the relevant clients [4, 2]. It does not claim to discover that derivation. Similarly, arbitrary-state completeness, cofinal encodings, and scheduler factoring have antecedents in the repository's checked-transport and further-interface proposals [6, 7].

Selected production proof sources	Before	After	Change
Lean implication pair, lines	811	401	−410
Lean implication pair, bytes	42,454	18,243	−24,211
Lean four enumerators + new core, lines	2,666	2,659	−7
Lean four enumerators + new core, bytes	109,663	109,339	−324
Rocq two scheduler clients + new core, lines	410	511	+101
Rocq two scheduler clients + new core, bytes	18,100	21,625	+3,525
Rocq implication proof + new core, lines	1,169	353	−816
Rocq implication proof + new core, bytes	22,673	17,449	−5,224

Table 1: Comments and whitespace are included. Each row has its stated source boundary; these are neither whole-repository sizes nor compiled proof sizes.

The complete six-file Lean implication integration, including proof data, renderer, producer and new adapter template, changes 80,277 to 55,874 bytes: 24,403 bytes saved. Audit programs, historical evidence, documentation, and the new standalone reproduction bundles are separate deliverable overhead. The source ledgers record every before/after hash, so these boundaries are reviewable rather than inferred from a hand-picked proof fragment.

1.2 Preserve the precise consistency theorem

The maintained Lean endpoint states [1]

```
BlankHalts machine215  ⟷  BProv ZFCax_s [] Form.fBot.
```

Its nonhalting equivalence is obtained by negating both sides. Thus the operative consistency predicate is syntactic non-refutability for the repository's chosen formalization of ZFC. The objective here is to shorten the proof of that equivalence while preserving the syntax, seed, program, compiler relation, and literal machine.

There are two different formalization obligations. The existing external proof-assistant endpoint establishes the displayed equivalence in its ambient formal environment. A theorem stated *inside ZFC*, or inside a weaker arithmetical metatheory, additionally needs the appropriate internal coding and derivability bridge. The repository itself explicitly leaves an internal PA endpoint open [3]. No component refactor in this packet fills that gap or asserts consistency or unconditional nonhalting. These distinctions should remain in the repository's theorem descriptions after integration.

2 Share the implication derivation at its weakest interface

The numeric side conditions obscure a simpler fact: the derivation only needs a relation that is reflexive and remains true when its right-hand object is embedded in either operand of implication. Arithmetic, parsing, truth values, and injectivity are unnecessary in this core.

Let A be a type with an operation $\Rightarrow : A \times A \to A$ and a relation $\preceq$ satisfying

$$\begin{aligned} a &\preceq a, \\ u \preceq x &\Longrightarrow u \preceq (x \Rightarrow y), \\ u \preceq y &\Longrightarrow u \preceq (x \Rightarrow y). \end{aligned}$$

For a provability predicate $T : A \to \mathsf{Prop}$, assume modus ponens and the restricted row

$$u \preceq a \quad \Longrightarrow \quad T(((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c))).$$

Write the formula in parentheses as $\rho_8(a, b, c, u)$. A single condensed-detachment graph supplies the following exports:

1. unrestricted ρ_8: $T(\rho_8(a, b, c, u))$ for all four arguments;
2. composition: $T((a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c)))$;
3. lifting under an antecedent: $T(a \Rightarrow b) \Rightarrow T((x \Rightarrow a) \Rightarrow (x \Rightarrow b))$;
4. K: $T(a \Rightarrow (b \Rightarrow a))$;
5. Peirce: $T(((a \Rightarrow b) \Rightarrow a) \Rightarrow a)$;
6. S: $T((a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c)))$.

The inherited proof-data file describes a 58-step derivation with six goals. Its Lean rendering has 56 named private lemmas, with the remaining final nodes inlined into public proofs. Its generator emits ordinary proof terms. The generic theory assumes neither transitivity of $\preceq$ nor implication injectivity. This is useful beyond cosmetic generality: it prevents numeric coding lemmas from becoming accidental dependencies of the propositional completeness bridge.

2.1 Two interpretations, one maintained proof graph

For the **numeric interpretation**, take the encoded implication operation and $u \preceq a$ to mean $u \leq a$. The existing pairing inequalities discharge the two growth properties. The concrete restricted calculus supplies modus ponens and its dominated ρ_8 rule. The adapter's unrestricted-row, composition and lifting theorems are then direct applications of the core. The Lean numeric adapter is byte-identical to the baseline in the supplied shared basis patch.

For the **unrestricted interpretation**, take ordinary implication syntax and define $u \preceq a$ to be True. All three relation obligations are trivial, and the unrestricted axiom supplies every restricted row. The existing `Lukasiewicz` namespace can keep its named results while deriving K, S, Peirce and composition from the same core. This changes `Lukasiewicz.lean` from 524 lines / 28,731 bytes to 101 lines / 4,097 bytes. The generic core grows modestly, from 287 to 300 lines, because its public interface now includes the three additional basis results.

Falsity remains a separate ingredient. The numeric and Tarski bridges still need their original falsity or ρ_6 assumptions. Pure implication alone is not being claimed to provide ex falso for an unrelated constant. Likewise, the existing `DomImp` interface should retain its truth-related field for compatibility

unless a separate consumer census justifies changing it. Its presence in the adapter does not force the generic derivation to use it.

2.2 Producer consistency is part of the refactor

Copying shorter Lean files without updating the producer would leave the repository in an unstable state: the next regeneration would restore the old proofs or fail a pinned-hash check. The supplied patch therefore changes six production paths together: the two Lean proof files, the maintained producer, the condensed-detachment renderer and six-goal proof data, and the small unrestricted adapter template. The producer preserves its existing `-out`, `-check`, and default command-line behavior.

Fresh producer replay reproduces all three maintained Lean outputs exactly. The four legacy renderer modes also reproduce their previous bytes. In particular, adding the shared Lean interpretation does not silently replace a different legacy Rocq rendering. The generator and its hash checks are useful engineering tools, but correctness comes from checking the emitted proofs in the assistant. No trust in the search program or renderer is added.

The Lean audit has fourteen explicitly typed old-interface clients. The six generic exports and unrestricted K, S, Peirce and composition are axiom-free. Numeric consumers retain `propext` and `Quot.sound`; the Tarski completeness consumers retain the standard classical profile. No new axiom or `sorryAx` appears in the component evidence.

2.3 Rocq implication component

The Rocq implementation consumes the exact same six-goal parser and proof data as the Lean shared-basis producer. It adapts the earlier generic Rocq proof interface from the repository's shared-proof-core proposal [5], replacing the maintained concrete certificate interpreter/replay with a generic theorem and a small numeric adapter. The generic module has 298 lines / 15,004 bytes, and the adapter has 55 lines / 2,445 bytes. Together they replace 1,169 lines / 22,673 bytes with 353 lines / 17,449 bytes: 816 lines and 5,224 bytes saved, including the entire new core. This is a 23.0% reduction in that selected proof source.

The adapter preserves all three original `DomImp` fields and the `rho8_all` contract. Its stronger `rho8_all_of_restricted` result also exposes that the truth field is unused by this particular implication theorem. The six generic exports, this numeric specialization, and the existing seed equivalence have nine closed assumption reports in the fresh combined compatibility build.

This optional integration patch follows the Rocq scheduler patch because both update the same producer and manifest. It adds a Rocq renderer and adapter template, updates generated sources and compile order, and depends on the shared six-goal proof data supplied by the Lean patch. A producer replay reproduces all five changed Rocq modules byte-for-byte. It exercises the calculus producer, not full machine regeneration.

With both Rocq refactors, the three old concrete proof modules total 40,773 bytes / 1,579 lines; the five replacement modules total 39,074 bytes / 864 lines. The net saving is 1,699 bytes (4.2%) and 715 lines. Template copies, generator code, and audit artifacts are outside this explicitly stated proof-module comparison.

3 Prove candidate scheduling once

The next duplicated mechanism is finite progress through an infinite sequence of finite candidates. Its proof should not know the formula representation, the stack discipline, or the compiler's command

constructors. It needs only a small progress interface and a separate adequacy lemma for a selected target.

3.1 Operational interface and progress laws

Let S be a state type and $\mathsf{step} : S \to \mathsf{Option}(S)$ a partial deterministic transition. Retain the existing concrete run function with equations

$$\mathsf{run}(0, s) = \mathsf{some}(s),$$
$$\mathsf{run}(n+1, s) = \begin{cases} \mathsf{run}(n, t), & \mathsf{step}(s) = \mathsf{some}(t), \\ \mathsf{none}, & \mathsf{step}(s) = \mathsf{none}. \end{cases}$$

Define halting by a finite successful prefix followed by an undefined step:

$$\mathrm{Halts}(s) \ :\Longleftrightarrow \ \exists n, t, \quad \mathsf{run}(n, s) = \mathsf{some}(t) \ \wedge \ \mathsf{step}(t) = \mathsf{none}.$$

Let $p, i : S \to \mathbb{N}$ be a pending-work rank and the candidate index. A boundary is a state with $p(s) = 0$. Require only these laws for successful steps $s \to t$:

$$p(s) = 0 \Longrightarrow i(t) = i(s) + 1, \tag{1}$$
$$p(s) \neq 0 \Longrightarrow p(t) < p(s) \ \wedge \ i(t) = i(s). \tag{2}$$

There is *no bound on the rank loaded by a boundary step.* In particular, the proof does not require $p(t) \leq i(t)$, monotone encodings, candidate injectivity, or an explicit time budget. The rank is a progress measure, not necessarily a literal instruction count.

Theorem 3.1 (Reach the target boundary or halt first)**.** *For any s and K with $i(s) \leq K$, either* $\mathrm{Halts}(s)$ *or there exist n, t such that*

$$\mathsf{run}(n, s) = \mathsf{some}(t), \qquad p(t) = 0, \qquad i(t) = K.$$

Proof. Use lexicographic induction on $(K - i(s), p(s))$, retaining $i(s) \leq K$. If $p(s) = 0$ and $i(s) = K$, use the zero-step run. Otherwise inspect the step. An undefined step proves halting immediately. At a successful active step, (2) preserves the first component and strictly decreases the second. At a successful boundary step below the target, (1) strictly decreases the first component and preserves the bound $i(t) \leq K$; the new second component may be arbitrarily large. Apply the induction hypothesis and prepend the step to its finite witness. □

Lean implements this as `reach_or_halt` with an explicit lexicographic termination measure. Rocq implements `seek` by nested strong induction using an equivalent nonnegative offset. The argument is constructive; the Rocq component's assumption audit is closed under the global context.

3.2 Use a local completion theorem with the correct quantifiers

Theorem 3.2 (Completion at a fixed reachable boundary)**.** *Suppose $i(s) \leq K$ and*

$$\forall n, t, \quad \big(\mathsf{run}(n, s) = \mathsf{some}(t) \wedge p(t) = 0 \wedge i(t) = K\big) \implies \mathrm{Halts}(t).$$

Then $\mathrm{Halts}(s)$.

Proof. Apply Theorem 3.1. Its halting alternative already gives the result. Otherwise the hypothesis gives halting from the reached boundary. Concatenate the successful prefix with that boundary's halting witness. □

This is the supplied `complete_at` interface. It requests adequacy only for boundary payloads actually reachable from the given start, which can be strictly weaker than adequacy for all states. The target K is selected before the universal quantification over those payloads. One useful stronger corollary assumes cofinally many indices that work for every boundary payload:

$$\forall N\ \exists K \geq N\ \forall t, \quad p(t) = 0 \wedge i(t) = K \Longrightarrow \mathrm{Halts}(t).$$

That condition yields halting from every state. It is more than the local interface requires and should be kept as a convenience corollary, not built into the kernel's record.

3.3 Instantiate the actual K215 enumerator

For `EnumPerm`, the pending proof-list code and next-candidate index give the two ranks. Existing concrete transition facts establish (1)–(2). The concrete transition and run definitions remain unchanged. Lean packages their equations in a `Dynamics` record; Rocq's generic iterator is definitionally equal to the maintained iterator. Neither needs a second semantic simulation proof.

Given a refutation in `C222`, the existing dominated-calculus compiler produces a command word P. For an arbitrary initial state s, choose $Y = i(s)$ and the one candidate

$$K = \mathtt{encL}(\mathtt{ax}\ \ 0\ 1\ Y,\ P +\!\!+ [\mathtt{drop}, \mathtt{nip}, \mathtt{nip}]).$$

The pairing argument `encL_ge` proves $Y \leq K$. The reset prefix and `program_none` make the selected complete word halt for every register payload supplied at that boundary. Therefore this single K discharges `complete_at`. No search for a candidate adapted to an unknown future payload is needed. The prefix clears the working formula and stack fields needed by the compiled contract; it need not reset every register to a fixed constant. The word's contract already accepts the remaining arbitrary payload [8].

The new Lean exports are, schematically,

```
complete_any : C222 F -> HaltsFrom s
halts_iff_any : Inv (toW s) -> (HaltsFrom s <-> C222 F)
```

The old boundary-restricted exports remain as compatibility wrappers. Rocq adds `enum_complete_from` and `enum_from_iff`, preserving the old enumerator and seed endpoints.

Keep the soundness invariant. Completeness can start from an arbitrary payload because the compiled witness resets what it needs. Soundness still requires the initial formula/cache values to be legitimate. An arbitrary malformed state might already trigger a halt without representing a ZFC refutation. Accordingly, the generalized equivalence retains `Inv`; the generalized completeness direction does not require it. Earlier repository work, including `EnumSafeStart254`, already demonstrates related arbitrary-state completeness [9, 6].

4 A checked counterexample protects the interface

A tempting replacement hypothesis is

$$\forall \text{ payload } b\ \forall N\ \exists K \geq N, \quad \text{candidate } K \text{ halts when started with payload } b.$$

This does not suffice if the payload can change between candidates. It puts the existential target on the wrong side of a universal quantifier.

The packet includes `QuantifierCounterexample.lean`, a standalone, axiom-free proof of the failure. States are pairs (k, b) with $k \in \mathbb{N}$ and $b \in \{\mathsf{false}, \mathsf{true}\}$. Define $\pi(0) = \mathsf{false}$ and $\pi(k+1) = \neg\pi(k)$, and set

$$\mathsf{step}(k, b) = \begin{cases} \mathsf{some}(k+1, \neg b), & b = \pi(k), \\ \mathsf{none}, & b \neq \pi(k). \end{cases}$$

Every state is a boundary: $p(k, b) = 0$ and $i(k, b) = k$. Thus successful steps satisfy exactly the scheduler's boundary-advance law; the active-step law is vacuous.

For each fixed b and lower bound N, one of $N, N+1$ has parity different from b, so there is a halting candidate beyond every bound for that fixed payload. Nevertheless the actual aligned trajectory never halts:

$$\mathsf{run}(n, (k, \pi(k))) = \mathsf{some}(k+n, \pi(k+n)).$$

At every candidate the payload has changed to the unique nonhalting parity. No index kills both possible payloads. The source checks payload-wise cofinality, aligned nonhalting, and absence of a uniform killing candidate, with empty axiom profiles for all three main results.

Index	Actual payload	Immediate halting payload	Actual next state
0	false	true	$(1, \mathsf{true})$
1	true	false	$(2, \mathsf{false})$
2	false	true	$(3, \mathsf{true})$
3	true	false	$(4, \mathsf{false})$

This is not a criticism of the maintained K215 compiler: its reset word provides the stronger, correct quantifier order. The counterexample explains why that reset/adequacy fact must remain explicit when the scheduler is made generic. It also prevents a future abstraction from replacing a uniform target with superficially similar pointwise cofinality.

5 Integration choices and their actual savings

5.1 Lean: share the induction and keep the concrete interface

The patch instantiates the scheduler in `EnumPerm`, `Enum254`, `Enum222`, and `EnumDom`. Each retains its familiar `run_inv`, `run_add`, `drain`, and `reach` interfaces. Their proofs invoke the shared kernel. The new kernel has three inductive proofs: run composition, preservation of an invariant along a run, and reach-or-halt. Halting transport and completion are short consequences. This replaces sixteen separately maintained inductive arguments across the four modules by three.

The static source audit preserves 117 original executable declarations and 237 original public theorem statements, comparing declaration text after comment/whitespace normalization. This is evidence about that explicitly defined comparison, not a universal semantic-equivalence checker. Actual proof compilation and typed clients provide the complementary checks. The existing `generate_enum222.py` and `generate_enumdom.py` remain unchanged and regenerate the delivered variants byte-for-byte from the refactored `Enum254`.

The resulting 113-line shared kernel offsets almost all of the removed client text. Across all four clients, the net reduction is seven lines / 324 bytes. For `EnumPerm` alone plus the new core, the selected source

group grows by 95 lines / 3,411 bytes. The value is a single place to maintain fairness, fewer proof copies, and a stronger reusable interface. It would be misleading to describe this step as a large K215 source-size reduction.

5.2 Rocq: preserve compatibility and synchronize regeneration

The scheduler component includes `CandidateScheduler.v`, a revised `L1_CompPerm.v`, and a shorter `SeedPerm.v`. The old `enum_complete`, `seedPerm_complete`, `seedPerm_sound`, and `seedPerm_iff_thm` types are preserved; generalized arbitrary-state results are added. Compatibility helpers remain available to downstream proofs.

The patch updates the generated sources, their `rocqperm` templates, `regenerate215.py`, and the extension manifest's hashes and compile order. Source-only changes without these producer updates would be incomplete. The two old clients shrink from 18,100 to 17,009 bytes, but the 4,616-byte core raises the selected total to 21,625 bytes. For a single lane this is an increase, and the small concrete rank proof remains a reasonable alternative.

There are two routes to later savings: reuse the kernel in additional enumerators, or retire now-redundant helper wrappers after a whole-consumer audit. That audit must include generated sources and templates. Preserving compatibility in this proposal makes its effect easier to review; it is not a claim that every wrapper deserves permanent retention.

5.3 Do not mix unrelated size measurements

Source length, endpoint import closure, elaboration time, proof-object size, and trusted assumptions measure different things. The packet reports source counts and scoped compilation evidence. It does not infer faster endpoint builds from shorter source or smaller proof objects from fewer lines.

In particular, the baseline's 179-module closure is already the result of earlier dependency factoring. The unrestricted `Lukasiewicz` shrinkage is valuable within that closure, but importing a new scheduler can add a module even as it removes duplicated induction. Recalculate the closure after adoption rather than subtracting the four-client aggregate savings from the K215 endpoint's byte count.

6 Validation, reproduction, and adoption

6.1 What was actually checked

The Lean component checks use Lean 4.32.0, compiler commit

`8c9756b28d64dab099da31a4c09229a9e6a2ef35`.

They need only the bundled `Init`/`Std` libraries and the included project providers. They do not require Mathlib. The shared-basis check freshly compiled fourteen provider/project modules plus a typed audit. The scheduler check freshly compiled its six new/refactored/audit modules over fourteen freshly checked providers. The changed proof files and audit clients emitted no warnings; the unchanged `TarskiComplete` provider emitted six existing linter warnings, retained in the logs.

The scheduler's generic `run_invariant` is axiom-free. Lean's generic reach and completion results report `propext` and `Quot.sound`; the concrete audited endpoints use only the existing `propext`, `Classical.choice`, and `Quot.sound` profile. The quantifier counterexample's three audited results are

axiom-free. These profiles are measured results of these builds, not a claim that every Lean encoding of the mathematical argument needs those axioms.

The Rocq scheduler check freshly compiled eleven dependency/component modules and a typed client, then passed a recursive `coqchk`. Six assumption reports were closed under the global context. The compiler available for this experiment was Coq 8.18.0 / OCaml 4.14.1. Build copies replace `From Stdlib` with `From Coq`; original source bytes and both hashes are retained. The current production lane specifies Rocq 9.1.0 with rocq-stdlib 9.0.0. Consequently this is a checked compatibility build, not a fresh claim about that designated toolchain or the whole physical machine endpoint.

The Rocq providers are the maintained numeric calculus and enumerator sources. The two older K230 arithmetic files are retained in the repository's checked-transport packet and match their production origins. They are not the stub providers from a generic-core demonstration. The source receipts make that distinction independently inspectable.

A final combined Lean build freshly compiled all 22 supplied modules in one isolated tree. It uses the restored six-goal basis, its rebuilt numeric provider, all four scheduler clients, fourteen shared-basis typed clients, ten scheduler typed clients, and the quantifier counterexample. No compiled project object was reused. All modules passed; only the six existing `TarskiComplete` warnings remained. This checks the interaction between the two refactors, without extending the claim to the full machine endpoint.

The final combined Rocq check freshly compiled twelve dependency/component modules and a typed client, then passed recursive `coqchk`. Its nine audited implication/adapter/seed results were closed under the global context. The self-contained driver's separate fresh run confirmed the same result using the included original source bytes and the documented compatibility copies.

Finally, all four production patches were applied in order to an isolated minimal baseline. The integration check matched thirteen checked proof-source hashes and reproduced three Lean basis outputs, two enumerator variants and five Rocq outputs exactly. The actual repository sources remained unchanged. This validates source correspondence and producer integration as well as the separate proof checks.

6.2 How to review the archive

The top-level `README.txt` lists exact commands and patch order. `components/` contains the focused implementation packets, with their source overlays, generators or adapters, audits, source measurements, and accepted verification logs. `lean/` contains a standalone combined Lean source bundle and a fresh-build driver; `rocq/` provides the analogous standalone Rocq bundle. `integration/` contains the combined patch and producer check. `evidence/` contains the pinned baseline endpoint inventory and integration checks. The archive manifest binds the delivered files by SHA-256. Compiled proof objects and failed development experiments are excluded.

The portable scripts accept explicit compiler and repository paths. They create isolated output directories and do not require writing into a ProveIt checkout. Logs preserve the actual commands and execution paths used in this experiment; portable instructions are separate from those historical records. Rebuilding the PDF needs an ordinary LaTeX installation with the packages declared at the top of this source.

6.3 Suggested adoption sequence

1. Start from the pinned revision, or rebase the small patches and repeat their source/producer comparisons against the desired target.

2. Adopt the shared implication basis and its producer together. Check the old public theorem types and regenerate all maintained outputs.

3. Decide whether the scheduler's cross-candidate reuse justifies its small immediate size cost for the intended branch. If so, apply its concrete adapters and producer/template changes together.
4. Build the combined affected components from fresh objects. Check their axiom profiles and reject any newly admitted lemma or unverified provider shortcut.
5. Run the repository's designated Lean and Rocq acceptance gates for affected endpoint descendants and both old endpoint clients. Record new source/object bindings; retain old checkpoint evidence as history.
6. Only after a consumer census, remove unused compatibility helpers or old generated derivations. Measure the actual endpoint closure and build characteristics at that point.

No completed source patch should be represented as already certified by an older whole-endpoint receipt. Conversely, reproducing every expensive machine site proof is not necessary to assess a clearly labeled component proposal. The appropriate integration gate supplies that final assurance at adoption.

7 The next larger simplification: route before tape

After the two checked refactors, the next useful boundary is the decoder certificate. The current proof already has extensive site coverage; the phase-twelve checkpoint retains evidence for 6,176 guarded outcomes [2]. The earlier further-interface report proposes choosing a finite route and a clock before the tape semantics [7]. The opportunity is to package a finite checked decoder path once and reuse a generic execution theorem, reducing repeated operational reasoning at leaves.

A suitable target statement has the quantifier order

$$\forall \text{ source path } p, \quad \exists \text{ route } r,\ k, \quad \forall \text{ admissible tapes } \tau, \quad \mathsf{run}(k, \mathsf{entry}(p, \tau)) = \mathsf{primitiveEntry}(r, \tau).$$

This is a design sketch, not a theorem supplied by this packet. The exact state relation and tape transformation must come from the maintained semantics. In particular, a proposition of the form $\forall \tau\, \exists k\, P(\tau, k)$ does not itself justify extracting one tape-independent k. Construct the route and its length from decidable source-path data, then prove the uniform execution theorem.

Keep the full routed-entry witness and terminal source bit. A physical state or a short bit suffix alone need not identify the semantic role. The clock should stop at primitive entry; primitive execution and the return scan have their own contracts and costs. This prevents an internal decoder bound from being mistaken for a whole-command running-time bound.

Primitive/return suffix factoring remains open at the pinned checkpoint. It should be a separate proposal with all guarded outcomes rerun, including the exception and exit cases. There is no justification for deleting site obligations merely because many outcomes look similar. For auxiliary lanes with repeated ad hoc halting budgets, the repository's reachable-suffix induction proposal is another promising reusable interface, provided it respects the actual register-equivalence relation rather than adding function extensionality [7].

The staged plan therefore keeps each review focused: first eliminate a duplicated implication derivation; then consolidate search progress; then factor finite decoder execution certificates. The supplied code makes the first two mechanisms concrete and gives the third a precise proof obligation.

A Minimal interfaces for maintainers

The actual Lean scheduler record retains the client's executable definitions:

```
structure Dynamics (S : Type u) where
  step : S -> Option S
  run  : Nat -> S -> Option S
  zero : forall s, run 0 s = some s
  succ : forall n s, run (n + 1) s =
    match step s with
    | some t => run n t
    | none   => none
```

Here ASCII arrows are used only for presentation. See the included `CofinalScheduler.lean` for the directly compilable source. The scheduler laws are separate from the dynamics so ordinary run composition and invariant transport do not unnecessarily depend on any rank.

The completion API, again in schematic notation, is:

```
complete_at (s : S) (target : Nat)
  (bound : index s <= target)
  (hit : forall n t,
    run n s = some t ->
    pending t = 0 -> index t = target -> Halts t)
  : Halts s
```

This is the principal client interface. Exposing an auxiliary proof of the lexicographic ranking as a requirement on each client would reintroduce the duplication the abstraction is intended to remove.

B Source references

Repository references below are pinned to the reviewed revision except for the explicit historical commit. The generated validation receipts in the archive are the primary evidence for the new component checks and source measurements.

References

[1] ProveIt, *KEndpoint215.lean*, exact halting and nonhalting equivalences.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean

[2] ProveIt, *Twelfth K215 proof-simplification checkpoint*, source closure, checking scope, and remaining work.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase12/README.md

[3] ProveIt, *K215 proof-simplification overview*, including the external/internal endpoint distinction.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/README.md

[4] ProveIt, historical shared-basis and producer work, commit `49e99c5b7cc6c2e164331961de28bad3904361fd`.
https://github.com/VladimirReshetnikov/ProveIt/commit/49e99c5b7cc6c2e164331961de28bad3904361fd

[5] ProveIt, *Shared proof-core proposal*, earlier representation-independent implication and adapter sources.
https://github.com/VladimirReshetnikov/ProveIt/tree/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/shared-proof-core

[6] ProveIt, *Checked transport / Rocq seed-rank packet*, earlier concrete rank and arbitrary-state completeness work.
https://github.com/VladimirReshetnikov/ProveIt/tree/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/checked-transport

[7] ProveIt, *Further interface proposal*, scheduler extraction, decoder route/clock separation, and suffix induction.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/further-interface/report.tex

[8] ProveIt, *EnumPerm.lean*, maintained enumerator, compiler, and completion proof at the baseline.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/EnumPerm.lean

[9] ProveIt, *EnumSafeStart254.lean*, earlier arbitrary-state completeness interface.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/EnumSafeStart254.lean

[10] ProveIt, *rocqperm/SeedPerm.v*, retained source for the seed-completeness adapter.
https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/rocqperm/SeedPerm.v